Self-Attention Step by Step

Contents

1. Overview . . . 2
2. Prerequisites . . . 2
3. The idea: a word as a mix of its sentence . . . 3
4. Weights from similarity . . . 4
    4.1 Dot products as similarity . . . 4
    4.2 Softmax turns scores into weights . . . 4
    4.3 The whole computation for one word . . . 5
5. All words at once, with matrices . . . 6
6. The problem: nothing to learn . . . 7
7. Three roles: query, key and value . . . 8
8. Query, key and value vectors from learned matrices . . . 10
    8.1 A linear transformation per role . . . 10
    8.2 The refined computation . . . 11
    8.3 All words at once . . . 12
9. Learned matrices on real reviews . . . 15
    9.1 The experiment . . . 15
    9.2 Results . . . 15
    9.3 What the weights learned . . . 16
10. Summary . . . 18
11. Sources . . . 18
12. Key terms . . . 19

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Attention mechanism; Contextual embeddings.
- **Leads to:** Scaled dot-product attention; Positional encoding; Masked self-attention.
- **Compare with:** Cross-attention.

1. Overview

Key point: Self-attention (G-1763) builds each word's new vector as a **weighted sum** (each vector multiplied by its weight, then added) of the embeddings (vectors, lists of numbers) of all the words in the sentence. The weights come from **dot products** (how similar two words are), normalised by a **softmax** (G-1830; it turns any numbers into positive weights that sum to 1). Each embedding plays three roles, **query**, **key** (G-1011) and **value** (G-2068), so three learned matrices (tables of numbers) W_Q, W_K, W_V (G-41) turn it into three vectors, one per role. For a whole sentence at once: $Y = \text{softmax}(QK^\top)V$ (built step by step in section 8).

Self-attention was treated as a box: static embeddings go in, **contextual embeddings** (G-462) come out. This Note opens the box. We build self-attention from first principles, as if inventing it:

1. a simple version with no parameters (sections 3 to 5);
2. its two properties: it runs in parallel, and it cannot learn (section 6);
3. the fix: query, key and value vectors from learned matrices (sections 7 and 8);
4. a test on real movie reviews: fixed against learned (section 9).

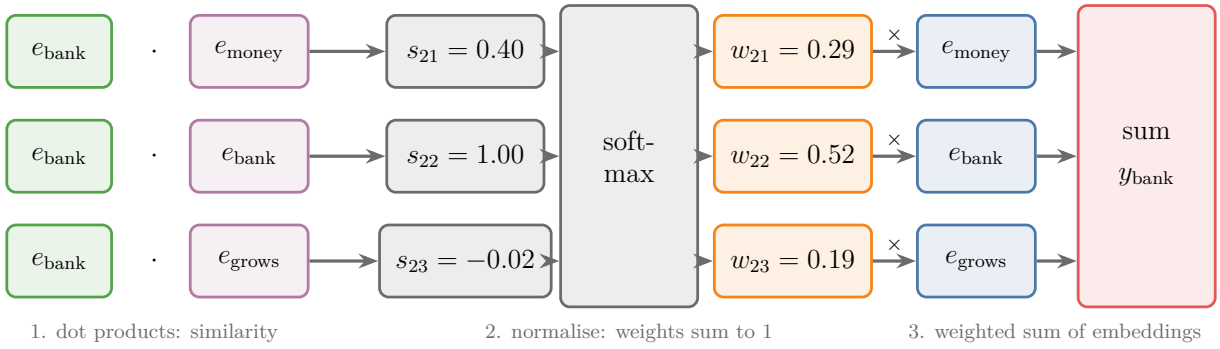


Figure 1: The first-principles computation of the new vector for "bank" in "money bank grows", with the real numbers of the Notebook: dot products, softmax, weighted sum

2. Prerequisites

- Static and contextual embeddings.

- The dot product and cosine similarity: the **dot product** (G-634) as a measure of similarity.
- The softmax formula: softmax turns any numbers into positive weights that sum to 1.
- Matrix multiplication as composition and **linear transformations** (G-1097): a vector times a matrix is a new vector.

3. The idea: a word as a mix of its sentence

> **Key point:** Write each word's new meaning as a mix of all the words of its sentence. "Bank" in "money bank grows" becomes mostly bank, some money, a little grows.

Take two short phrases, "money bank grows" and "river bank flows". "Bank" means something different in each, but a static embedding gives it the same vector in both. Suppose we describe each word not by itself but as a mix of the words around it:

$$\text{bank}_{\text{new}} = 0.29\,\text{money} + 0.52\,\text{bank} + 0.19\,\text{grows}$$
$$\text{bank}_{\text{new}} = 0.25\,\text{river} + 0.54\,\text{bank} + 0.21\,\text{flows}$$

The left-hand sides are the same word, but the right-hand sides differ, because the neighbours differ. Here each word stands for its vector (a list of numbers). The meaning of "bank" now depends on its context (Figure 2).

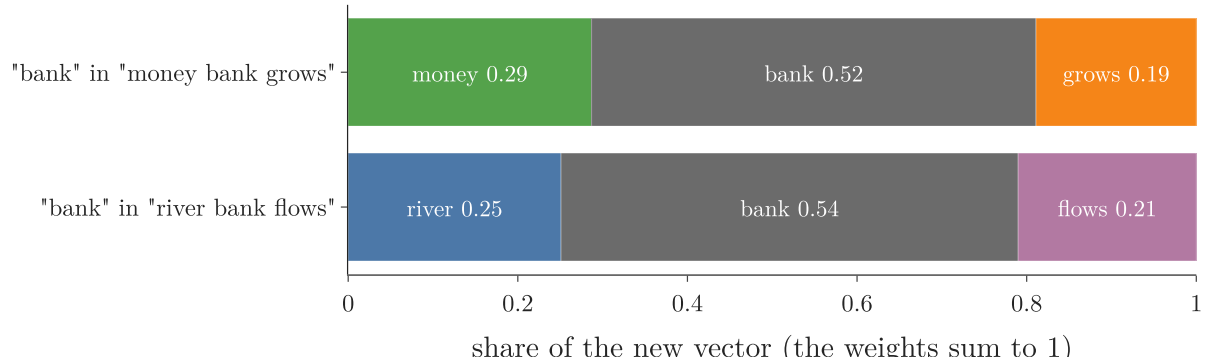


Figure 2: The new "bank" in each phrase as a mix of the words of the phrase, with the real weights of section 4. The same word keeps about half of itself and borrows the rest from different neighbours

We can do the same for every word of each phrase: "money" becomes a mix of money, bank and grows, and so on.

Machines work with vectors, not words, so we write the mix with embeddings. With e_{money} the embedding of "money" (a vector of n numbers), the new vector of "bank" is

$$y_{\text{bank}} = 0.29\,e_{\text{money}} + 0.52\,e_{\text{bank}} + 0.19\,e_{\text{grows}}$$

A weighted sum of n-number vectors is again an n-number vector. The new vectors are the **contextual embeddings** (G-462) we wanted. Two questions remain: where do the weights come from, and how do we make them sum to 1?

4. Weights from similarity

> **Key point:** A weight says how much one word should borrow from another, so it should measure how similar the two words are. The **dot product** (G-634) of their embeddings measures similarity; the softmax turns the dot products into weights that sum to 1.

4.1 Dot products as similarity

> **Key point:** Two vectors that point the same way have a large dot product.

The weight 0.29 means: "bank" takes 29% of its new meaning from "money". The more related two words are, the larger that share should be. The **dot product** (G-634) of two vectors is a simple measure of how similar they are (see the geometric meaning).

1. **In words:** multiply the vectors number by number, then add.
2. **Formula:**
$$a \cdot b = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$
3. **Example:** with $a = (6, 1)$, $b = (4, 2)$ and $c = (1, 5)$:
$$a \cdot b = 6 \times 4 + 1 \times 2$$
$$a \cdot b = 26$$
$$a \cdot c = 6 \times 1 + 1 \times 5$$
$$a \cdot c = 11$$
a is more similar to b than to c, and the dot products say so.

In the Notebook every embedding has length 1, so the dot product is exactly the **cosine similarity** (G-491; the cosine of the angle between two vectors): 1 for a word with itself, near 0 for unrelated words.

4.2 Softmax turns scores into weights

> **Key point:** Raw **dot products** (G-634) can be any size and even negative. The softmax makes them positive and makes them sum to 1.

Call $s_{ij} = e_i \cdot e_j$ the **score** (G-223, attention score) of word i with word j. Scores can be large, small or negative, for example 36, -15 and 32. We want weights (the **attention weights**, G-225) that are positive and sum to 1, so they read as shares: "bank is 52% bank, 29% money, 19% grows". The **softmax** function does exactly this.

1. **In words:** raise e (Euler's number, about 2.718) to the power of each score, then divide by the total.

2. **Formula:** for word i in a sentence of N words,

$$w_{ij} = \frac{e^{s_{ij}}}{e^{s_{i1}} + e^{s_{i2}} + \cdots + e^{s_{iN}}}$$

3. **Example:** the real scores of "bank" (word 2) in "money bank grows", from the Notebook's embeddings, are $s_{21} = 0.40$ (with money), $s_{22} = 1.00$ (with itself) and $s_{23} = -0.02$ (with grows):

$$e^{0.40} = 1.49$$

$$e^{1.00} = 2.72$$

$$e^{-0.02} = 0.98$$

$$\text{total} = 1.49 + 2.72 + 0.98 = 5.19$$

$$w_{21} = \frac{1.49}{5.19} = 0.29$$

$$w_{22} = \frac{2.72}{5.19} = 0.52$$

$$w_{23} = \frac{0.98}{5.19} = 0.19$$

The weights sum to 1, and they are the numbers of section 3.

4.3 The whole computation for one word

> **Key point:** Three steps: **dot products** (G-634) with every word, softmax, weighted sum.

Figure 1 puts the steps together for "bank":

1. **Scores:** the dot product of e_{bank} with each embedding of the sentence gives s_{21}, s_{22}, s_{23}.
2. **Weights:** the softmax of the scores gives w_{21}, w_{22}, w_{23}, which sum to 1.
3. **Output:** the weighted sum of the embeddings:

$$y_{\text{bank}} = w_{21}\, e_{\text{money}}$$

$$+\ w_{22}\, e_{\text{bank}}$$

$$+\ w_{23}\, e_{\text{grows}}$$

The same three steps, with "money" and then "grows" in place of "bank", give y_{money} and y_{grows}. This simple version of attention, scores by dot product, softmax, weighted sum, is the one SLP3 (§7.1) presents before adding parameters.

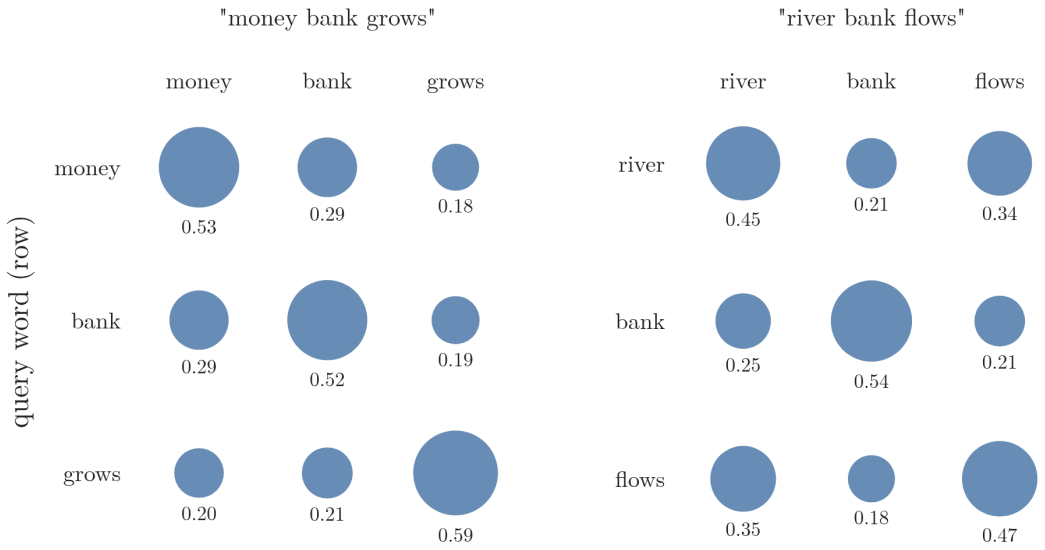


Figure 3: The weights of the simple self-attention for both phrases, with real embeddings, as a grid of dots: the area of each dot is the weight. Row i shows how word i mixes the words of its phrase; every row sums to 1

Figure 3 shows all the weights, one dot per pair of words, so the large ones stand out at a glance (a picture of the weights used by Sanderson 2024, Ch 6). Each word keeps about half of its own meaning (the diagonal: a vector is most similar to itself) and borrows the rest from its neighbours. The row of "bank" differs between the two phrases: it borrows 0.29 from "money" in one, 0.25 from "river" in the other. These outputs move "bank" towards "river" in "river bank flows" (shown in self-attention: static in, contextual out).

5. All words at once, with matrices

> **Key point:** Stack the embeddings as the rows of a matrix X. Then $XX^\top$ holds every score, a row-wise softmax gives every weight, and multiplying by X gives every output: $Y = \text{softmax}(XX^\top)\,X$.

Nothing in the computation for "grows" waits for the computation for "bank": each word's new vector needs only the embeddings, which are all known at the start. So all the words can be processed at the same time, using matrices.

1. **In words:** put the embeddings of the N words as the rows of a matrix; multiply it by its own transpose (the same matrix with rows and columns swapped, written $X^\top$) to get all the scores; apply the softmax to each row; multiply the weights by the embeddings.
2. **Formula:** with X of shape $N \times n$ (one row per word),

$$S = XX^\top \quad (N \times N)$$

$$W = \text{softmax}_{\text{rows}}(S)$$

$$Y = WX \quad (N \times n)$$

Entry (i, j) of S is $e_i \cdot e_j = s_{ij}$, and row i of Y is y_i.

3. **Example:** for "money bank grows", X is 3×50 (3 words, 50 numbers each), S and W are 3×3 (Figure 3, left) and Y is 3×50. For "i put my money in the bank", X is 7×50, S is 7×7 and Y is again 7×50. In the Notebook, the matrix version gives exactly the same numbers as a loop over the words, one at a time. Figure 4 shows the shapes.

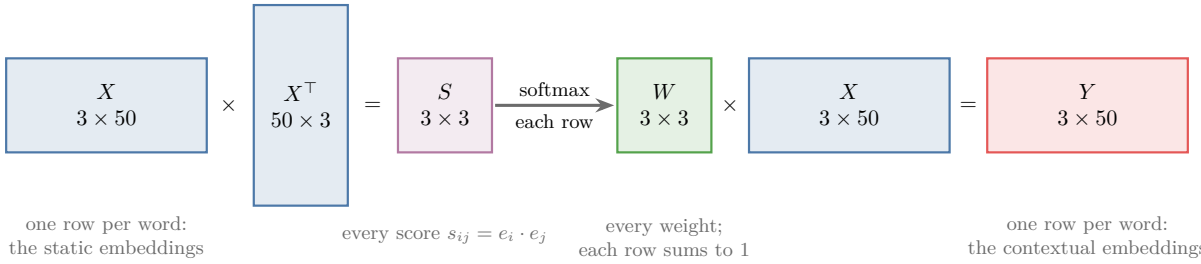


Figure 4: The whole sentence at once, for "money bank grows". One product gives every score, a softmax on each row gives every weight, and a second product gives every contextual embedding

Whether the sentence has 3 words or 3,000, the work is a few matrix multiplications, which a GPU does in parallel. This parallelism is what lets transformers train fast (measured in parallel versus sequential).

The parallel computation has a cost. Nothing in $Y = \text{softmax}(XX^\top)X$ depends on the order of the rows: shuffle the words and each word gets the same new vector, only in a different row. The order of words is lost, although word order matters in text. The transformer restores it separately, with **positional encoding** (G-1528; adding a vector that tells each word's position).

6. The problem: nothing to learn

> **Key point:** The simple version has no parameters. Its **contextual embeddings** (G-462) are general: the same for every task and every dataset. A task often needs **task-specific** contextual embeddings, so we must add learnable weights.

The three steps are a **dot product** (G-634), a softmax and another product. None of them has a weight or a bias. With no parameters there is nothing to train: the output for a sentence depends only on the static embeddings, never on the task or its data.

The resulting contextual embeddings are **general** (G-836, general contextual embeddings): they use the context, but in the same way for every task. Sometimes that is not enough. Suppose we train an English-to-Hindi translator whose data contains

- "how are you" → "main theek hoon" (as a reply), and

- "that task is a piece of cake" → "vah kaam bahut aasaan hai" ("that work is very easy").

General contextual embeddings of "piece of cake" lean towards the literal meaning, a slice of cake. A good translator must learn from its data that the phrase means "very easy". In the same way, "break a leg" should become "shubh kaamnaayein" ("good wishes") and not a sentence about a broken leg. What the model needs are **task-specific contextual embeddings** (G-1952): still built from the neighbouring words, but in a way learned from the task's data. For that, **self-attention** (G-1763) needs learnable parameters.

Where can they go? The softmax is a fixed formula. The parameters can only enter in the two products: the one that computes the scores and the one that forms the weighted sum.

7. Three roles: query, key and value

> **Key point:** In the computation, every embedding is used three times, in three roles: as the **query** that asks "how similar are you to me?", as the **key** that is compared with the query, and as the **value** that is mixed into the output.

Look again at Figure 1. The embedding of "bank" appears in three different places:

- **Query** (green): e_{bank} asks every word of the sentence, "how similar are you to me?". It is the word whose new vector we are computing.
- **Key** (purple): e_{money}, e_{bank}, e_{grows} answer: their **dot product** (G-634) with the query gives the scores. In the computation for "money" or "grows", e_{bank} also serves as a **key** (G-1011).
- **Value** (blue): the same embeddings are mixed with the weights to form the output.

The names come from dictionaries in programming. A Python dictionary `d = {"a": 2, "b": 3, "c": 4}` stores **keys** ("a", "b", "c") with their **values** (2, 3, 4). The lookup `d["a"]` is a **query**: it is compared with every key, and the value of the matching key, 2, is returned. **Self-attention** (G-1763) is a soft version of this lookup: the query is compared with every key, and instead of returning one value, it returns a mix of all the values, weighted by how well each key matches. SLP3 (§7.1) describes the same three roles.

The problem with the simple version is that one vector, the static embedding, plays all three roles. An analogy shows why that is a poor design. A writer is looking for a partner on a matrimonial website. His full life story is in his 300-page autobiography: his embedding. The website needs three things from him:

- a **profile** that others see when they search (his key);
- a **search** describing whom he is looking for (his query);
- what he says once there is a **match** (his **value** (G-2068)).

Uploading the whole autobiography as profile, search and first message would work badly in all three places. Instead he writes three short, focused texts, each drawn from the autobiography but shaped for its job. And he learns from experience what to put in each. If he gets matched with people who love politics when he does not, he removes "I write about politics" from his

profile; if a broad search works better than a narrow one, he widens it. Each version is tuned from the results, that is, from data.

Self-attention should do the same: from each embedding, derive three separate vectors, a query vector, a key vector and a value vector, and learn from the training data how to derive them.

What a query and a key could compute: an invented example. Suppose the model would like nouns to take in the adjectives in front of them, as in "the old brown dog". Imagine that each noun's query asks "are there adjectives before me?" and that each adjective's key answers "I am an adjective here". If the matrices put such questions and answers into the same direction, a large dot product means a match (Sanderson 2024, Ch 6, who builds the same kind of imagined example). With 2 numbers per query and key:

1. **In words:** the query of "dog" points along the first axis (looking for adjectives); the keys of "old" and "brown" point mostly the same way; the keys of "the" and "dog" point along the second axis.
2. **Formula:** $s_{\text{dog},j} = q_{\text{dog}} \cdot k_j$, then the softmax over j.
3. **Example:** $q_{\text{dog}} = (3, 0)$, $k_{\text{the}} = (0, 2)$, $k_{\text{old}} = (3, 0.5)$, $k_{\text{brown}} = (2.8, 1)$, $k_{\text{dog}} = (0.2, 2)$. The scores are 0, 9, 8.4 and 0.6, and the softmax gives the weights 0.000, 0.646, 0.354 and 0.000: "dog" takes its new meaning from "old" and "brown" only (Figure 5).

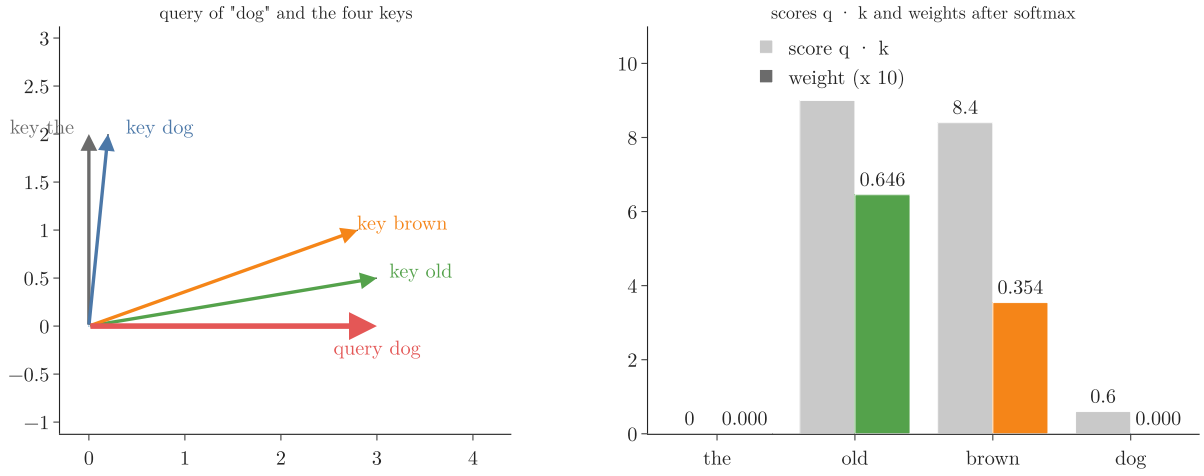


Figure 5: The invented example. Left: the query of "dog" (red) points along the first axis, like the keys of "old" and "brown"; the keys of "the" and "dog" point along the second. Right: the scores and the weights after the softmax

In a trained model nobody chooses these directions, and what a matrix actually learns is much harder to read; the example only shows the kind of job a query and a key can do together. Section 9.3 shows a real one: learned matrices that make every word look for the sentiment word.

8. Query, key and value vectors from learned matrices

> **Key point:** Multiply each embedding by three matrices: $q = eW_Q$, $k = eW_K$, $v = eW_V$. The same three matrices serve every word. They start random and are trained by backpropagation, so the **contextual embeddings** (G-462) become task-specific.

8.1 A linear transformation per role

> **Key point:** A new vector from an old one: multiply by a matrix.

To make a new vector from an old one, scaling alone (making it longer or shorter) is too limited: the direction must change as well. The standard way is a **linear transformation** (G-1097), multiplying the vector by a matrix (see **linear transformations** (G-1097)). We use three matrices, W_Q, W_K and W_V, one per role.

1. **In words:** each embedding, written as a row vector, times each of the three matrices gives the word's query, key and **value** (G-2068) vectors.
2. **Formula:**

$$q_i = e_i W_Q$$
$$k_i = e_i W_K$$
$$v_i = e_i W_V$$

   which is the form of SLP3 (eq. 7.9).
3. **Example:** in 2-D, with $e = (1, 2)$,

$$W_Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$
$$W_K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$
$$W_V = \begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}$$
$$q = (1 \cdot 1 + 2 \cdot 1,\ 1 \cdot 0 + 2 \cdot 1)$$
$$q = (3, 2)$$
$$k = (1 \cdot 0 + 2 \cdot 1,\ 1 \cdot 1 + 2 \cdot 0)$$
$$k = (2, 1)$$
$$v = (1 \cdot 2 + 2 \cdot 0,\ 1 \cdot 0 + 2 \cdot 0.5)$$
$$v = (2, 1)$$

   One embedding has become three different vectors (Figure 6; here k and v happen to be equal, because W_K and W_V map this particular e to the same point).

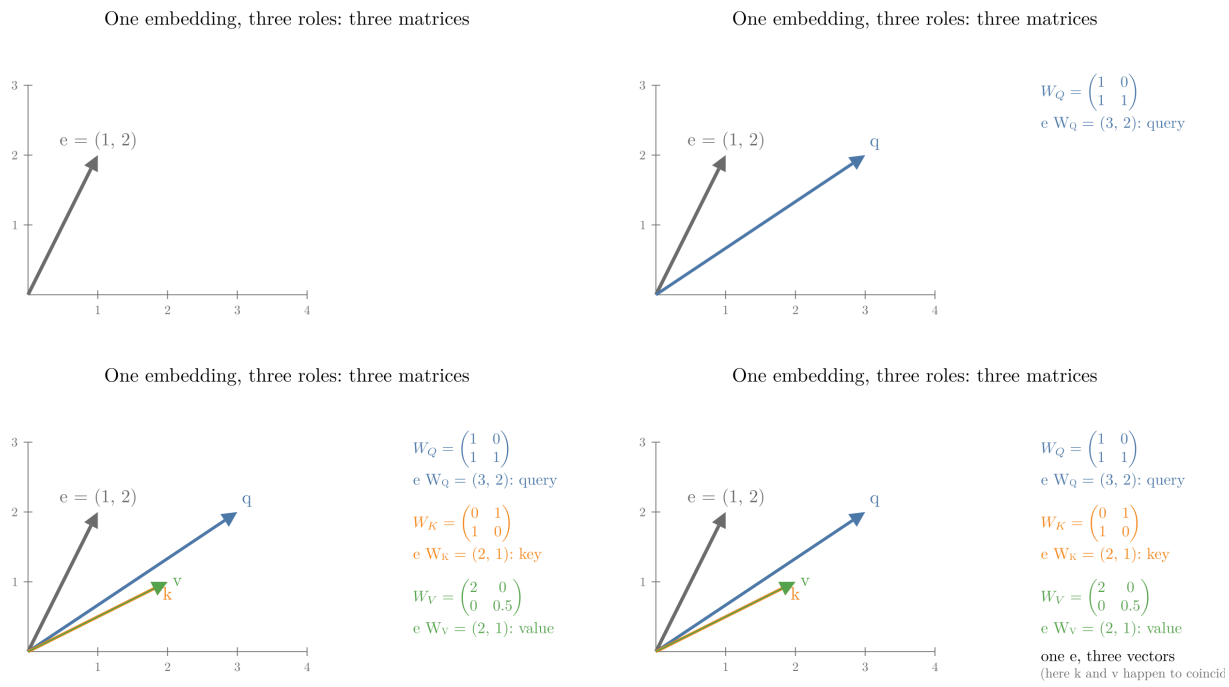


Figure 6: The embedding $e = (1, 2)$ multiplied by W_Q, W_K and W_V: each matrix moves the arrow to a new place

The same matrices for every word. The W_Q used for "money" is exactly the W_Q used for "bank" and for "grows", number for number, and the same holds for W_K and W_V. Three words give 9 vectors from only three matrices:

$$3 \times 3 = 9$$

Learned from data. Nobody chooses the numbers in W_Q, W_K and W_V. They start random. The model translates (or classifies) a sentence, makes mistakes, and backpropagation changes the matrices to reduce the loss; the next sentence gets slightly better query, key and value vectors. At the end of training the matrices hold the numbers that produce the best vectors for this task.

8.2 The refined computation

> **Key point:** Same three steps as before, with queries in the dot product, **keys** (G-1011) as what they are compared with, and values in the weighted sum.

The computation of section 4 stays the same; each use of an embedding is replaced by the vector for its role. For "bank":

$$s_{2j} = q_{\text{bank}} \cdot k_j$$

$$w_{2j} = \text{softmax}_j(s_{2j})$$

$$y_{\text{bank}} = \sum_j w_{2j}\, v_j$$

8.3 All words at once

> **Key point:** $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and then $Y = \text{softmax}(QK^\top)\,V$. Still fully parallel.

The matrix form of section 5 carries over (Figure 7, and SLP3 eq. 7.33):

1. **In words:** multiply the embedding matrix by the three weight matrices; compare every query with every **key** (G-1011) in one product; softmax each row; mix the values.
2. **Formula:**

$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$
$$Y = \text{softmax}(QK^\top)\,V$$

3. **Example:** for "money bank grows" with 50-number embeddings and 50×50 matrices: X is 3×50; Q, K and V are 3×50; $QK^\top$ is 3×3, one score for every pair of words; Y is 3×50. The Notebook checks that the matrix form equals the word-by-word loop.

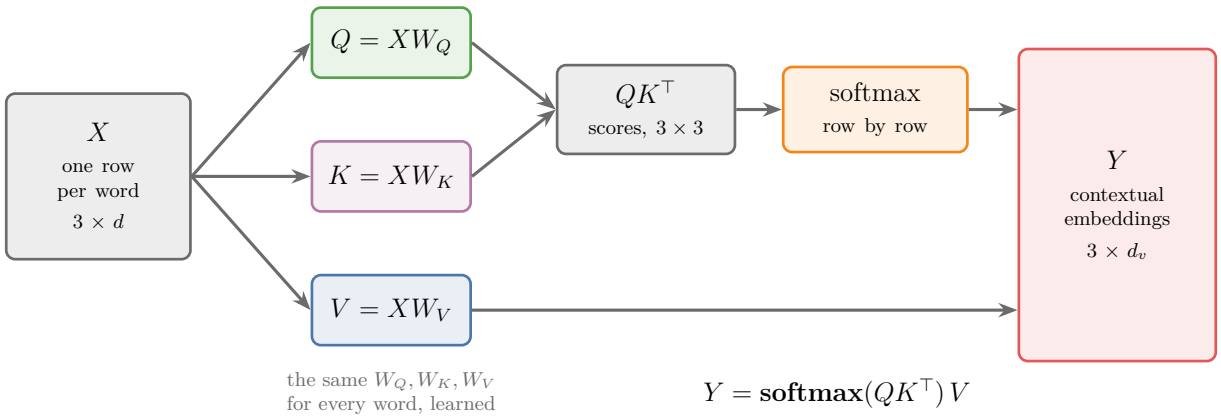


Figure 7: Self-attention with learned matrices, for a whole sentence at once

The same steps with every number. Give the three words invented embeddings of 2 numbers, and reuse the matrices W_Q, W_K, W_V of section 8.1. Figure 8 runs the computation, one matrix operation per frame.

The rows of X are money, bank, grows:

$$X = \begin{pmatrix} 1 & 2 \\ 2 & 0 \\ 0 & 1 \end{pmatrix}$$

1. **Queries, keys, values:** each row of X times each matrix.

$$Q = XW_Q = \begin{pmatrix} 3 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}$$

$$K = XW_K = \begin{pmatrix} 2 & 1 \\ 0 & 2 \\ 1 & 0 \end{pmatrix}$$

$$V = XW_V = \begin{pmatrix} 2 & 1 \\ 4 & 0 \\ 0 & 0.5 \end{pmatrix}$$

2. **Scores:** every query with every key. For the query of "money", $(3, 2)$, the score with each key is:

$$\text{money: } 3 \times 2 + 2 \times 1 = 8$$

$$\text{bank: } 3 \times 0 + 2 \times 2 = 4$$

$$\text{grows: } 3 \times 1 + 2 \times 0 = 3$$

$$QK^\top = \begin{pmatrix} 8 & 4 & 3 \\ 4 & 0 & 2 \\ 3 & 2 & 1 \end{pmatrix}$$

3. **Weights:** the softmax of each row. For the row of "grows", $(3, 2, 1)$:

$$e^3 = 20.09$$

$$e^2 = 7.39$$

$$e^1 = 2.72$$

$$\text{total} = 30.19$$

$$\text{weights} = \frac{20.09}{30.19}, \frac{7.39}{30.19}, \frac{2.72}{30.19}$$

$$\text{weights} = 0.67,\ 0.24,\ 0.09$$

All rows together:

$$W = \text{softmax}(QK^\top)$$

$$W = \begin{pmatrix} 0.98 & 0.02 & 0.01 \\ 0.87 & 0.02 & 0.12 \\ 0.67 & 0.24 & 0.09 \end{pmatrix}$$

4. **Output:** each row of weights mixes the rows of V. For "grows":

$$0.67 \times (2, 1) = (1.34,\ 0.67)$$

$$0.24 \times (4, 0) = (0.96,\ 0)$$

$$0.09 \times (0, 0.5) = (0,\ 0.045)$$

$$\text{sum} = (2.31,\ 0.71)$$

All rows together:

$$Y = \begin{pmatrix} 2.02 & 0.98 \\ 1.80 & 0.93 \\ 2.31 & 0.71 \end{pmatrix}$$

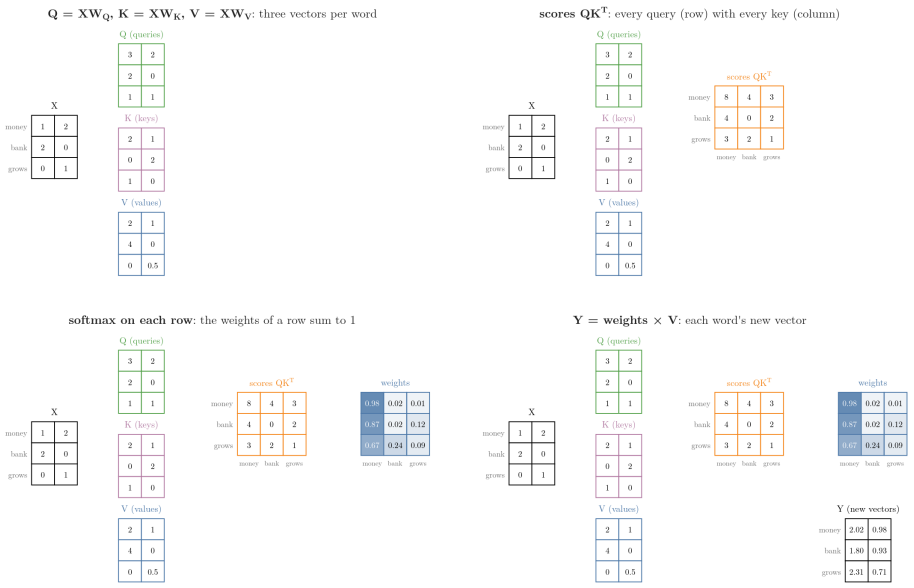


Figure 8: The worked example, one operation per frame: X; then Q, K, V; the scores $QK^\top$; the weights after a softmax on each row (darker = larger); and Y, the weights times V (a full numeric walk-through after StatQuest, "The matrix math behind transformer neural networks, one step at a time!!!"; the numbers are our own)

In Figure 8, watch the row of "money" in the weights: a score of 8 against 4 and 3 already takes 0.98 of the weight, because the softmax works with e to the power of the score. Large scores make the softmax nearly all-or-nothing, which is the problem the next Note solves.

The order matters: $QK^\top$, queries times the transpose of the keys, so that entry (i, j) is $q_i \cdot k_j$.

The **self-attention** (G-1763) inside a transformer works this way, with one more step: the scores are divided by $\sqrt{d_k}$ before the softmax. Why that scaling is needed is explained in why a wide spread is a problem. Inside a transformer, Y also does not replace X: it is added to X, so attention supplies a change to each word's vector (the residual connection of **add and norm** (G-172)).

9. Learned matrices on real reviews

> **Key point:** On IMDB movie reviews, with the same frozen embeddings, **self-attention** (G-1763) with learned W_Q, W_K, W_V reaches 0.84 test accuracy against 0.69 without parameters. The learned weights concentrate on the word that matters for the task: "terrible".

9.1 The experiment

> **Key point:** Two models identical except for the three matrices, which start at the identity, so both begin as the same function.

Section 6 claimed that learnable matrices make the **contextual embeddings** (G-462) task-specific. The Notebook tests the claim on sentiment analysis of IMDB movie reviews (the **IMDB dataset** (G-923)): 25,000 training and 25,000 test reviews, each an **observation** (G-1374; one record of the data), with the sentiment, positive or negative, as the **target** (G-1949; the output we predict). The words are the 10,000 most frequent, and each review is cut to 200 words.

- **Static embeddings:** 50 numbers per word, learned from the training reviews themselves by the counting method of the same effect in real text, then **frozen**, so they cannot adapt to the task. They do capture meaning: the nearest words to "great" are wonderful, fine, fantastic, excellent, marvelous, superb.
- **Model:** embeddings → **self-attention** (G-1763) → the average of the contextual embeddings over the words of the review → one sigmoid node.
- **Fixed:** self-attention without parameters, $Y = \text{softmax}(XX^\top)X$. The only trainable weights are the 51 of the final node.
- **Learned:** $Y = \text{softmax}(QK^\top)V$ with W_Q, W_K, W_V of size 50×50, which adds 7,500 trainable weights. The three matrices start at the **identity matrix** (G-915; ones on the diagonal, zeros elsewhere, so it leaves a vector unchanged), so at the start the learned model computes exactly what the fixed model computes.

Both train with Adam (a variant of gradient descent) for 5 epochs (full passes over the training reviews), batch size 64 (reviews per weight update), 3 runs each with different seeds.

9.2 Results

> **Key point:** The learned model is about 15 points more accurate, in every run.

Model	Trainable weights	Test accuracy (mean of 3 runs)
Fixed: no parameters	51	0.69
Learned W_Q, W_K, W_V	7,551	**0.84**

The three runs of each model agree to within 0.004. Both models have exactly the same information: the same frozen embeddings and the same reviews. The only difference is that

the learned model can change how each word looks for, and mixes in, the other words, based on the labelled reviews.

9.3 What the weights learned

> **Key point:** Without parameters, each word mostly attends to itself. After training, almost every word attends to "terrible".

What did the matrices learn? Figure 9 takes a real 10-word stretch from a test review, "and the rest of the cast rendered terrible performances the", and shows the attention weights of both models.

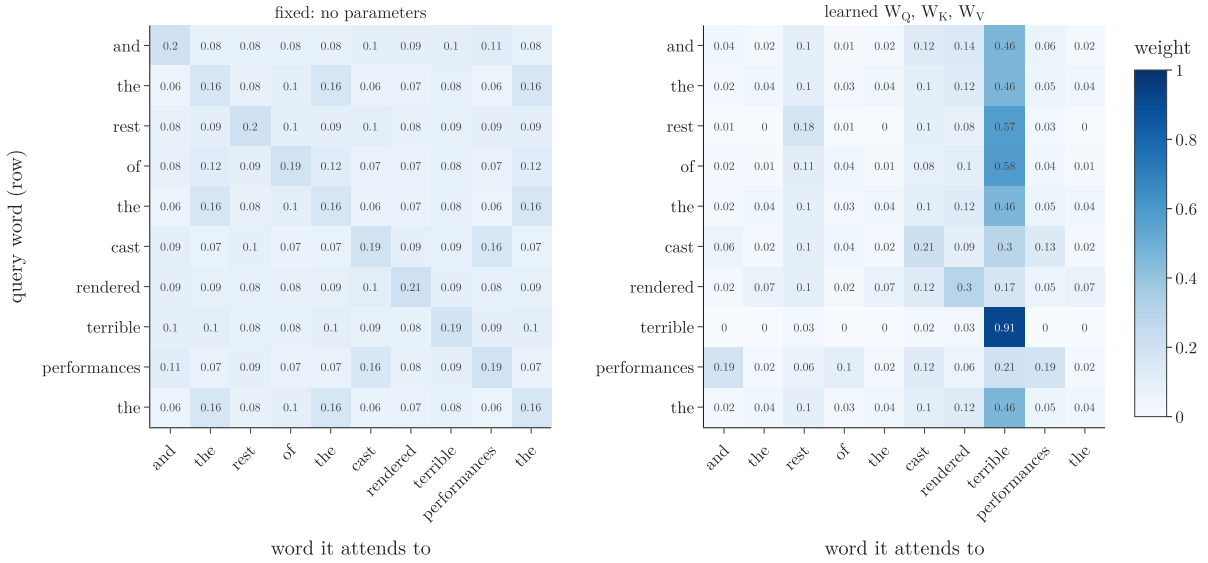


Figure 9: Attention weights on a real review. Left: without parameters, each word borrows mostly from itself and spreads the rest evenly. Right: with learned W_Q, W_K, W_V, almost every word borrows most from "terrible", the word that decides the sentiment

- **Fixed (left):** the largest weight in each row is on the diagonal, about 0.2: each word is most similar to itself. The rest is spread evenly, about 0.08 per word. The weights know nothing about sentiment.
- **Learned (right):** the column of "terrible" lights up. "Rest" puts 0.57 of its weight on "terrible", "of" 0.58, "the" 0.46, and "terrible" itself 0.91. After averaging, the review's vector is dominated by the meaning of "terrible", which is exactly what a sentiment classifier needs.

For this task, the learned matrices turned **self-attention** (G-1763) into a detector of sentiment words. A translation model trained on the same embeddings would learn different matrices, and attend differently. These are the task-specific contextual embeddings of section 6, measured.

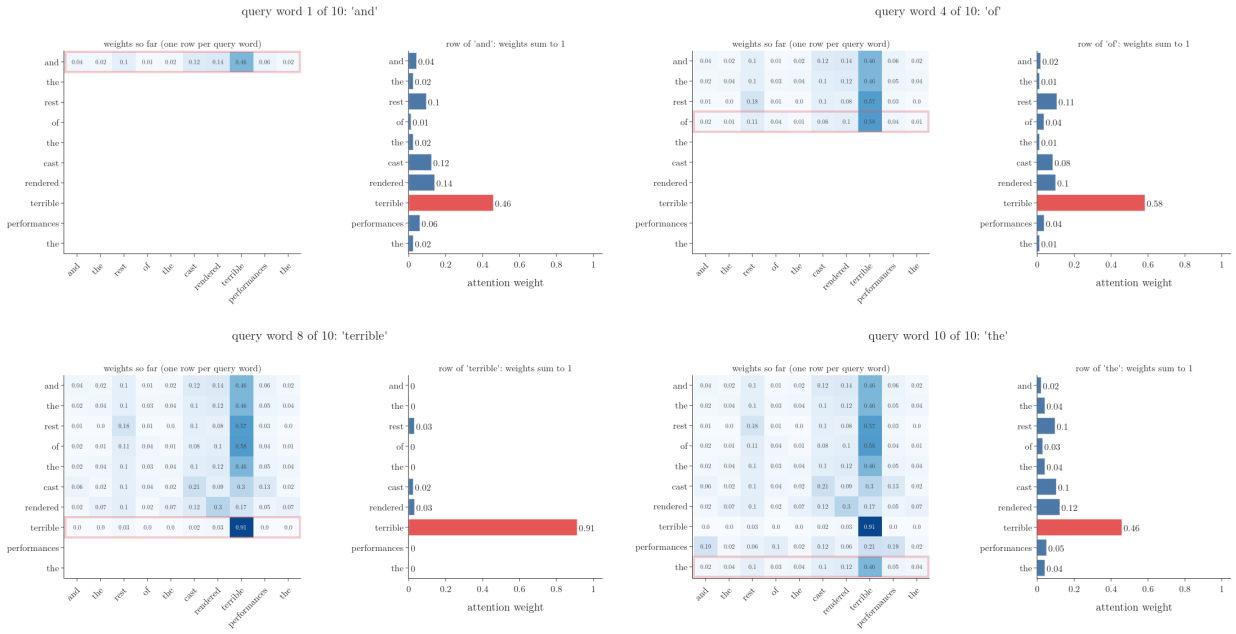


Figure 10: The learned weights forming one query word at a time. Left: the weight matrix filling row by row. Right: the current query word's weights, which sum to 1; the largest is red

10. Summary

Step	Simple version	Refined version (used in transformers)
Vectors per word	the embedding e_i, used in all three roles	query $q_i = e_i W_Q$, key $k_i = e_i W_K$, value $v_i = e_i W_V$
Scores	$s_{ij} = e_i \cdot e_j$	$s_{ij} = q_i \cdot k_j$
Weights	softmax over j	softmax over j
Output	$y_i = \sum_j w_{ij}\, e_j$	$y_i = \sum_j w_{ij}\, v_j$
Whole sentence	$Y = \text{softmax}(XX^\top)\, X$	$Y = \text{softmax}(QK^\top)\, V$
Parameters	none: general contextual embeddings	W_Q, W_K, W_V, learned: task-specific

- A word's **contextual embedding** (G-462) is a weighted sum of the vectors of all the words in its sentence, so the same word gets a different vector when its neighbours differ.
- The weights are similarities (**dot products** (G-634)) normalised by a softmax, so each row sums to 1 and reads as shares: related words lend each other more.
- All words are processed at once with matrix products, because no word's new vector waits for another's; this runs in parallel on a GPU but ignores word order, which positional encoding has to restore.
- Learned matrices W_Q, W_K, W_V, the same for every word, make the result task-specific, because without them there is nothing to train: on IMDB, 0.84 accuracy against 0.69, with attention concentrated on sentiment words.
- The transformer adds one more step, dividing the scores by $\sqrt{d_k}$, because large scores make the softmax nearly all-or-nothing (why the scores are scaled).
- This is what happens inside the **self-attention** (G-1763) box: dot products, softmax and a weighted sum turn static embeddings into contextual ones.

11. Sources

Built from

- CampusX, "Self Attention in Transformers | Deep Learning | Simple Explanation with Code!", YouTube, https://www.youtube.com/watch?v=-tCKPl_8Xb8
- StatQuest with Josh Starmer, "The matrix math behind transformer neural networks, one step at a time!!!", YouTube, https://www.youtube.com/watch?v=KphmOJnLAdI. 04:30–12:00 (self-attention written out as matrices with every number: queries, keys and values, the scores, the row softmax, the weighted values).
- Sanderson, G. (3Blue1Brown), "Attention in transformers, step-by-step | Deep Learning Chapter 6", 2024, 3blue1brown.com/lessons/attention, https://www.youtube.com/watch?v=eMlx5fFN 5:52–10:03 (an imagined head in which nouns' queries look for preceding adjectives and adjectives' keys answer; the attention pattern drawn as a grid of dots).

Other references

- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026), web.stanford.edu/~jurafsky/slp3. Chapter 7, section 7.1 (simplified attention: dot-product scores, softmax, weighted sum, eqs. 7.6–7.8; the three roles query, key and value and the matrices W_Q, W_K, W_V, eq. 7.9); section 7.3 (all positions at once: $Q = XW_Q$, $K = XW_K$, $V = XW_V$ and $QK^\top$, eq. 7.33 and Figure 7.9).
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Section 3.2 (queries, keys and values; the output is a weighted sum of the values).
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y. and Potts, C. (2011). Learning Word Vectors for Sentiment Analysis. *ACL 2011*. The IMDB review dataset.
- Tatoeba (tatoeba.org) English–French sentence pairs, prepared by manythings.org/anki and distributed for the Keras examples. Licence CC BY 2.0. Source of the embeddings of sections 3 to 5.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Key (G-1011)	In attention, the vector each word offers to be compared with a query; the dot product of the query with a key scores how much attention that word gets.
Value (G-2068)	In self-attention, the vector each word contributes to the weighted sum that becomes a word's new vector; it comes from the word's embedding through a learned value matrix.
W_Q, W_K, W_V (G-41)	The learned matrices that turn an embedding into its query, key and value vectors; the same for every word.
Attention score (G-223)	A number that measures how strongly one word relates to another: the dot product of the first word's query with the second word's key. The softmax turns one word's scores into its attention weights.
Attention weight (G-225)	How much one word draws on another word when building its new vector: an attention score after the softmax, so the weights of one word are non-negative and sum to 1.
General contextual embedding (G-836)	A word vector that changes with the surrounding words but is computed without any learned parameters, so it is the same for every task.

Term	Meaning
Task-specific contextual embedding (G-1952)	A contextual embedding (a word's vector built from its neighbouring words) whose way of mixing the neighbours is learned from the task's data, so a phrase such as "piece of cake" gets the meaning the task needs.
Key and value (G-1010)	In attention, the vectors of the words being looked at: keys are compared with the query to get the weights, and values are mixed with those weights; in cross-attention both come from the encoder's input words.
Self-attention (intra-attention) (G-1763)	Attention in which the words of one sequence attend to each other (the queries, keys and values all come from that sequence); it gives each word a new vector that depends on the words around it.
Softmax function (G-1830)	A function that turns a list of scores into probabilities that add up to 1: raise e to each score and divide by the total, $e^{z_k} / \sum_j e^{z_j}$; it gives one probability per class in multi-class models.
Contextual embedding (G-462)	A vector for a word that changes with the other words of its sentence, so "bank" gets one vector in "river bank" and another in "money bank". Self-attention produces it.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^{\mathsf{T}}x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Linear transformation (G-1097)	A change of the whole plane by a matrix that keeps grid lines straight and evenly spaced, and the origin fixed; algebraically, $L(\mathbf{v}+\mathbf{w}) = L(\mathbf{v}) + L(\mathbf{w})$ and $L(c\mathbf{v}) = cL(\mathbf{v})$.
Cosine similarity (G-491)	A score from -1 to 1 for how closely two vectors point the same way: the cosine of the angle between them. It ignores their lengths, so it compares texts or word embeddings by direction only.
Positional encoding (G-1528)	A vector added to each word's embedding that tells the model the word's position.

Term	Meaning
Add and norm (G-172)	Adds a sub-layer's input back to its output (a residual connection) and then layer-normalises the sum, so the signal and gradients pass through deep stacks and the numbers stay on one scale: LayerNorm(x + Sublayer(x)).
IMDB dataset (G-923)	50,000 film reviews labelled positive or negative, a standard sentiment-analysis dataset.
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Identity matrix (G-915)	The matrix that leaves every vector unchanged.
Score (G-1753)	In naive Bayes, the likelihood times the prior for one class; it is not the probability of the class, only proportional to it, which is enough to pick the class with the largest score.
Query, key, value vectors (G-1607)	Three vectors made from each word's embedding by learned matrices: a word's query is compared with every word's key by dot products to get attention weights, and those weights mix the value vectors into the word's new vector.